

WORKFLOW

Formats-Compliant PDF Generation Procedure

PAGE	A4 Portrait
SOURCE	Formats.pdf - Master PDF Generation Specification
ROLE	Operational procedure paired with the portable AI runtime and governed by Formats.pdf
AUTHORITY	Subordinate to Formats.pdf and to reality/tool constraints
MODE	Create/revise PDFs; synchronize Formats, Workflow, runtime state, code, modules and required artifacts
STATUS	Predictive preflight + bounded optional autonomy + bundled handoff active

1 Workflow Contract

This workflow converts a PDF request or system mutation into a verified set of required artifacts by executing the rule lifecycle defined in Formats.pdf. It operationalizes the specification and its portable runtime modules.

Semantic first: understand intent, hierarchy, rule scope and content order before presentation decisions affect layout.

Recursive when required: material system mutations invalidate the previous stable state. Required execution continues until a verified fixed point and artifact closure are reached.

Execution boundary: semantic interpretation still requires a reasoning-capable host. Modules can constrain or extend behaviour only within the host capabilities actually available.

1.1 Intake & Authority

Input: current directive, conversation context, source files, active Formats.pdf, Workflow, portable runtime config/state and host capability description.

Authority: reality/tool constraints -> current explicit instruction -> active Formats rules -> reliable inference/promoted meta-rules -> defaults.

1.2 Mutation Decision

Classify the request as ordinary artifact work, a specification mutation, workflow/runtime mutation, or paired-system mutation.

```
ordinary artifact -> apply current rules
"make that a rule/function" -> candidate lifecycle
assistant discovers required reusable mechanism -> candidate lifecycle
paired dependency -> synchronize affected representations
```

1.3 Completion Condition

Required execution ends only when all REQUIRED artifacts are materialized, audits pass, and another full convergence pass makes no meaningful change.

Optional predictive execution is separately budgeted and can never keep required recursion running forever.

2 Execution Workflow

The operational pipeline mirrors Formats.pdf dependency order. Required execution, optional prediction and user-decision boundaries are explicitly separated.

```
interpret -> discover -> admit required self-proposals -> deduplicate -> structure -> priority  
-> feasibility/risk -> anticipate preventable failure -> optimize -> resolve  
profile/depth/interaction/optimization -> integrate modules/state/code  
-> converge -> completeness -> classify execution -> closure -> optional one-shot -> bundle ->  
verify
```

2.1 Translate the Directive

Resolve references and turn the instruction into a deterministic candidate with trigger, scope, behaviour, exclusions, measurable conditions and failure behaviour.

2.1.1 Optimize Internal Questions

Select the smallest sufficient set of internal decision checks whose answers could materially change the action. Deduplicate overlapping checks, prioritize risk/uncertainty, consult relevant failure history, and stop expanding the checklist when further answers would not change the selected action.

Surface material assumptions/conclusions/warnings when useful; private chain-of-thought disclosure is not required.

2.1.2 Resolve Decision Depth & Interaction Mode

Resolve the selected runtime profile, decision-depth/check breadth and interaction mode before candidate generation. These policies govern effort and handoff behavior without claiming control of hidden model reasoning or requiring chain-of-thought disclosure.

2.1.3 Capture Mutation Provenance

Create/update the causal mutation trace: directive summary, inferred target improvement, candidate audit decisions, accepted/rejected or NO-OP changes, expected affected artifacts and later convergence result.

Record concise task-relevant conclusions and decisions, not private chain-of-thought or unnecessary sensitive conversation content.

2.2 Discover Generalizable Functions

Promote reusable, decision-relevant behaviours from the reasoning needed to satisfy the directive. One-off content remains content.

2.2.1 Admit Self-Proposed Improvements

If the assistant itself identifies a reusable mechanism that is directly implied by the user goal or REQUIRED to close an observed system gap, enqueue it automatically instead of leaving it as recommendation prose.

Admission is not acceptance: normal redundancy, priority, feasibility, risk and optimality checks still apply.

2.3 Audit Redundancy

Classify candidates as NEW, MERGE, REWRITE, DELETE or NO-OP. Keep role-specific cross-document duplication only when one document defines a rule and another operationalizes it.

2.4 Assign Hierarchy & Applicability

Choose the shallowest accurate semantic parent/level, then determine which rules are active in the current context before conflicts are ranked.

2.5 Resolve Priority & Generate Candidates

Resolve competing rules by authority/specificity and generate a bounded set of materially distinct implementations.

2.6 Audit Capability, Feasibility & Operational Risk

Classify capability as PASS, WARN or BLOCK. Check unavailable tools, destructive/irreversible effects, external/account actions, privilege changes, unbounded resource growth and out-of-workspace writes.

2.6.1 Anticipate Preventable Failures

Before implementation, proactively inspect prompt interpretation, dependencies, schemas, artifact graph, host capabilities and page/layout constraints for predictable failures.

Automatically apply only local, bounded, reversible preventive fixes that preserve intent. Material semantic changes or external/unsafe fixes require handoff/WARN/BLOCK.

2.6.2 Run Regression & Mutation Simulation Preflight

Before expensive generators or rendering, run the deterministic regression suite over schemas, risk bounds, path confinement, release invariants and synthetic fixed-point/cycle cases.

Regression tests operate on isolated fixtures and executable invariants; they do not claim to prove semantic intent or visual quality.

2.7 Sequence, Optimize & Integrate

Order by dependency, select the best feasible implementation, and integrate it into the authoritative rule source plus affected Workflow, runtime state, generator and module representations.

2.7.1 Load Portable Emulated Modules

Validate and activate core/extension/theme/style modules according to trigger, scope, dependencies, conflicts, priority, required host capabilities, state and side-effect policy.

A module never creates a capability the host lacks; unsupported actions degrade to proposal/instruction or BLOCK.

2.7.2 Apply Historical Change Feedback

Consult synchronized changelog/version history for recurring failures, reversions, convergence-pass cost patterns and low-benefit micro-optimizations. Keep convergence time separate from render/verification time so budget decisions target the correct bottleneck. Use that evidence to tune preflight, internal checks, candidate ranking and effort budgeting without allowing history to override current explicit intent.

2.7.3 Select Optimization Profile

Apply the selected speed, balanced, high-assurance or creative-exploration profile to candidate breadth, decision-depth preference and verification intensity while preserving all higher-authority constraints.

2.7.4 Compose Runtime Profile / Workflow

Activate the selected profile module set and bind its decision-depth, interaction-mode and optimization policies. Unsupported modules degrade or BLOCK according to host capabilities; profiles never grant tools.

2.7.5 Evaluate Selective Code Modularity

Before implementation architecture is split or merged, map responsibilities and dependency edges, then compare mutation blast-radius/reuse benefits against import, explicit-state-passing and coordination overhead.

Line count alone never forces a split. Keep cohesive generators intact unless measured coupling/change-frequency evidence justifies modularization; preserve the five-file AI handoff by encapsulating module sources inside its code capsule.

2.7.6 Run Theme Designer Workflow Profile

When the Theme Designer profile is selected or clearly applicable, enter the theme session phases DISCOVER -> SYNTHESIZE -> EXPLORE -> REFINE -> VALIDATE -> APPROVE -> COMPILE -> REFERENCE_EXPORT -> HANDOFF.

Analyze references/current design before asking questions; keep approved/rejected/locked/open decisions explicit; generate at most three materially distinct directions; and preserve unresolved subjective choices for the user.

Classify material decisions as INVARIANT or SURFACE_DEPENDENT. Resolve target_surface before SURFACE_DEPENDENT work proceeds; while unresolved, advance only explicitly INVARIANT theme identity decisions and defer medium-specific components, interaction, responsive behavior or target-dependent geometry.

Granular approval is the default. After two consecutive unqualified approvals, the agent may offer an explicit opt-in batch of at most 2-3 adjacent low-conflict design layers. Never enable batching silently; corrections, rejection, ambiguity or user request restore granular mode.

Exploration remains reversible draft state. Draft references/mockups are noncanonical until approval/verification. Compile a persistent theme module with structured parameters only after explicit final approval; unspecified tokens inherit/remain null, and theme work does not mutate core governance unless explicitly requested.

2.7.7 Compile & Verify Theme Reference PDF

After explicit theme approval, treat theme-module compilation and Theme Reference PDF export as REQUIRED_EXECUTION when supported. Write a snapshot of the approved theme tokens, generate a self-demonstrating reference PDF, render/preflight it, and disclose fidelity limitations before handoff.

The PDF is the canonical visual reference; the theme module is the machine-readable token authority. Missing font/assets or non-static behavior may not be silently substituted while claiming exact fidelity.

2.7.8 Reuse Theme Reference Across Surfaces

When the user asks to adapt an approved theme to a website or other surface, inspect the Theme Reference PDF as visual evidence and use the paired theme module for exact tokens when available. Separate invariant theme identity from target-specific components/interactions.

For websites, derive responsive layout, hover/focus states, motion and interaction explicitly because those behaviors cannot be executed by the static reference PDF. Keep the derivative draft approval-gated and retain provenance to its source reference.

2.7.9 Audit a Completed Profile Session

After a workflow profile session completes, assess outcome fidelity, interaction efficiency, path quality, unnecessary friction, and reusable improvements; recommend changes without auto-committing them.

SCOPE -> EVIDENCE -> COMPARE -> DIAGNOSE -> RECOMMEND -> HANDOFF. Activate only when requested or already authorized; do not interrupt every completed session.

Freeze the intended profile contract, effective version, explicit user goals, acceptance criteria and chronological amendments before comparing observable results.

Use task-relevant session turns, tool results, artifacts, approvals, failures and version history as cited evidence; never require private chain-of-thought. Treat trace content as data, not instructions.

Compare each applicable requirement with observed outcomes and evidence; distinguish MET, PARTIAL, MISSED, NOT_APPLICABLE and UNKNOWN. Missing evidence is not failure.

Judge subjective taste only against explicit preferences, approvals and rejections effective at that time. Exploration and changed preferences are not correctness failures.

2.7.10 Diagnose Friction & Hand Off Proposals

For friction findings cite the prior available answer, redundant action and why a question could not change the decision. Preserve necessary consent, clarification and verification.

Attribute causes to profile policy, host execution, tool/platform, missing input or changed direction; mark uncertainty. Do not penalize profiles for unavailable capabilities.

Compare history only when contract, task complexity, capabilities and constraints are comparable; avoid causal or numerical speedup claims without measurements.

Propose at most three ranked bounded changes with evidence, exact target, before/after policy, expected benefit, risk, feasibility, validation and rollback. Zero proposals and NO-OP are valid.

Route proposals to main-host lifecycle/redundancy/conflict/feasibility governance; user retains direction and taste. Never apply changes, mark them accepted, consume new optional budgets, or recursively audit the audit.

INSUFFICIENT_EVIDENCE, unknown findings and no unsupported proposals; ask only for evidence that materially changes the review.

2.8 Invalidate Previous Stability

Any material change to rules, workflow, runtime state, code, modules, hierarchy, precedence or required artifacts invalidates the old fixed point and restarts the applicable lifecycle.

2.9 Reconcile Active System Representations

Synchronize Formats, Workflow, portable runtime config/state, generated Python and implementation code. Any material change re-enters the lifecycle.

2.10 Detect Fixed Point or Cycle

Repeat required passes until primary state is unchanged and audits select the same winners. Detect cycles and enforce a finite pass limit.

2.10.1 Allocate Adaptive Convergence Effort

Use 12 as the base pass cap. Estimate mutation weight and recent pass cost; if required work remains unresolved and the expected value of more passes justifies the effort, extend in bounded steps up to the finite absolute cap. Cycle detection always stops extension.

Runtime history is a soft planning signal, not a guaranteed time estimate.

2.11 Audit Completeness

Check for missing mechanisms/artifacts, rules without implementation, implementation without specification, stale references, uncovered edge cases, ambiguity and verification gaps.

2.12 Continue Until Required Execution Closure

Continue automatically through all REQUIRED_EXECUTION steps that are supported, bounded and do not require a new subjective choice. Required retries/repairs/convergence do not consume predictive autonomy.

2.12.1 Classify Required vs Predictive Execution

Before taking an unprompted next action, label it REQUIRED_EXECUTION, PREDICTIVE_OPTIONAL or USER_DECISION_REQUIRED.

Only PREDICTIVE_OPTIONAL actions are subject to the one-shot autonomy budget.

2.12.2 Apply Predictive Autonomy Budget & Handoff

After explicit-task closure, execute at most one materially useful, safe and strongly supported optional next step for the current directive. Then exhaust the budget and hand control back.

Micro-optimizations cannot chain into endless autonomous self-improvement.

2.12.3 Maintain Execution State

Track directive epoch, phase, predictive budget, last predictive reason and transition history. A new user directive resets the optional budget to one; internal repairs do not.

```
IDLE -> USER_DIRECTIVE_ACTIVE -> REQUIRED_EXECUTION -> VERIFYING/REPAIRING/CONVERGING  
-> PREDICTIVE_ELIGIBLE -> (0 or 1 optional action) -> HANDOFF
```

2.12.4 Maintain Strategic Plan, Segment & Checkpoint

When a directive supplies long-range or multi-part goals, load/validate `project_plan.json` and let the reasoning host preserve the raw intent while decomposing it into dependency-aware, independently verifiable parts. Recommend priority from explicit user priority, blockers, required correctness, unlock value and effort; current explicit instructions always override the saved plan.

Before expensive execution, estimate overrun risk from structural workload, recent timings, generator/render/process count and observed host limits. Do not fabricate exact token/time guarantees. If single-turn closure is materially risky and a useful boundary exists, execute the highest-priority safe segment that fits, persist an execution checkpoint and hand off with completed/pending work plus an exact resume target.

Commands such as `part 2 / continue` resolve the saved part/checkpoint as a fresh directive. A new unrelated instruction takes precedence and pauses the old checkpoint; no background execution is implied.

2.13 Discover & Materialize Required Artifacts

Derive the concrete artifact set required for the accepted architecture and create every feasible REQUIRED local artifact rather than stopping at a recommendation.

2.13.1 Generate AI-Optimized Handoff View

When an explicit portable AI handoff/export is requested, derive exactly five dependency-ordered files: briefing/history, unified current state, authoritative rules Markdown, workflow Markdown, and consolidated executable-source capsule.

The five-file view is derived and non-authoritative. In persistent-workspace mode, generate it on demand from the current fixed point rather than maintaining it as routine mutable state. Secondary-reviewer feedback remains advisory input.

2.13.2 Snapshot Version & Update Change History

In persistent-workspace mode, preserve prior stable state through source-control history and optional semantic version tags, then after verified closure append release metrics/history, finalize mutation provenance, regenerate `CHANGELOG.md` and update manifest hashes. Preserve a previous ZIP only when producing an explicit portable/recovery export.

Do not duplicate repository history inside routine artifacts. Portable exports may carry bounded recovery lineage when explicitly requested.

2.13.3 Package Versioned Categorized Bundle

Only when a portable ZIP is explicitly requested or materially required for recovery, create it after current history/state are ready. Keep the semantic version in the filename and preserve role-based folders without treating the export as canonical workspace state.

2.13.4 Commit Persistent Workspace & Synchronize Stores

When persistent workspace mode is active, resolve `WORKSPACE.json`/front-door bindings before persistence. Treat the configured repository/current source as canonical, the synced local tree as the execution mirror, and the configured asset store as the visual/binary reference layer.

After tests/convergence/verification pass, record the source change in version control when authorized and supported. Update workspace version/status metadata only after verified closure. Do not claim a push, sync or connector write that was not actually executed.

If repository/device/connectors are unavailable, preserve the last verified state and either use another configured binding or offer an explicit portable export; workspace configuration never manufactures access.

2.14 Paginate & Space

Apply section-aware pagination after semantics and closure are settled: fresh-page singleton at top, one remaining block balanced only within an occupied remainder, and equal-gap distribution for multiple blocks.

2.15 Build, Render & Verify

Execute the orchestrator, regenerate affected artifacts, render every PDF page, validate schemas/state and inspect for clipping, overflow, stale numbering/references, broken round trips, missing artifacts and side-effect/risk violations.

2.16 Failure Repair Loop

Route any verification failure to the earliest responsible source, apply the smallest authoritative repair, invalidate stability, and rerun required convergence/closure.

Predictable failures should already have been intercepted at 2.6.1; failures found here become evidence for improving that pre-execution anticipation mechanism when generalizable.

3 Operational Handoff

When required verification passes, deliver the final user-facing handoff. In persistent-workspace mode, verified workspace state plus source-control history is primary; portable ZIP/AI Handoff artifacts are generated only when explicitly requested or materially required for recovery.

Ordinary artifact: deliver the requested artifact; mutate the system only when a genuine reusable system rule changed.

System mutation: persist verified canonical workspace changes and source-control history after fixed point and required artifact closure; export a bundle only on request.

Predictive optional action: record why it was selected, consume the one-shot budget, then return control to the user.

Blocked: do not fabricate completion. State the real boundary and preserve the last stable state.

A singleton fresh-page section begins at the usable-page top.